From Gradient Descent to Machine Learning

Goal. Give an intuition-first, concise overview of gradient descent (GD) and how it powers learning with backpropagation. Suitable as a 1–3 page handout.

1. Motivation & Intuition

You want to minimize a loss function $E(\mathbf{w})$ over parameters $\mathbf{w}$. In many ML problems, we do not know the closed-form shape of E; we only know how to *evaluate* it on data. Think of standing on a foggy mountain: you cannot see the entire landscape, but you can feel the local slope and step downhill.

Steepest descent update. The gradient points to the direction of *steepest increase*:

$$\nabla E(\mathbf{w}) = \left(\frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_d} \right).$$

To decrease the loss, move in the opposite direction:

$$\boxed{\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \nabla E(\mathbf{w}_t)}$$

where $\eta > 0$ is the learning rate (step size).

Direction is not necessarily 45°. In 2D, the gradient direction angle is $\theta = \tan^{-1}\big((\partial E/\partial w_2)/(\partial E/\partial w_1)\big)$. Only when the two partials have equal magnitude does it align at 45°.

2. From Optimization to Learning

In supervised ML, the loss is a data average:

$$E(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} L\big(f(\mathbf{x}_i; \mathbf{w}),\, y_i\big),$$

where f is the model, L is a pointwise loss (e.g., MSE or cross-entropy). We typically cannot write E in a simple closed form, but we can *run* the model on a batch of examples to compute the loss and its gradient.

Training loop (mini-batch SGD).

1. Sample a mini-batch $\mathcal{B} \subset \{1, \dots, N\}$.
2. **Forward:** compute predictions $\hat{y}_i = f(\mathbf{x}_i; \mathbf{w})$ and loss $E_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} L(\hat{y}_i, y_i)$.
3. **Backward:** compute $\nabla_{\mathbf{w}} E_{\mathcal{B}}$ via backpropagation.
4. **Update:** $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} E_{\mathcal{B}}$.

3. Backpropagation in One Picture & a Few Equations

Consider a tiny 2-layer network:

$$\begin{aligned} \mathbf{z} &= W\mathbf{x} + \mathbf{b}, \\ \mathbf{a} &= \sigma(\mathbf{z}), \\ \hat{y} &= \mathbf{v}^\top \mathbf{a} + c, \\ E &= \tfrac{1}{2}(\hat{y} - y)^2. \end{aligned}$$

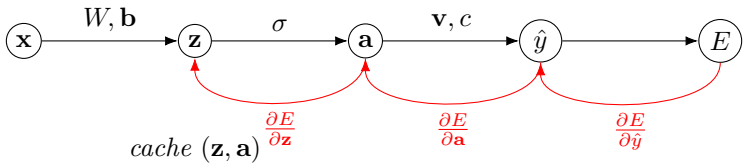
Backprop computes sensitivities using the chain rule, reusing cached forward values $(\mathbf{z}, \mathbf{a})$:

$$\frac{\partial E}{\partial \hat{y}} = (\hat{y} - y), \qquad \frac{\partial E}{\partial \mathbf{v}} = \frac{\partial E}{\partial \hat{y}}\, \mathbf{a}, \qquad \frac{\partial E}{\partial c} = \frac{\partial E}{\partial \hat{y}}.$$

$$\frac{\partial E}{\partial \mathbf{a}} = \frac{\partial E}{\partial \hat{y}}\, \mathbf{v}, \qquad \frac{\partial E}{\partial \mathbf{z}} = \left(\frac{\partial E}{\partial \mathbf{a}}\right) \odot \sigma'(\mathbf{z}),$$

$$\boxed{\frac{\partial E}{\partial W} = \left(\frac{\partial E}{\partial \mathbf{z}}\right) \mathbf{x}^\top, \quad \frac{\partial E}{\partial \mathbf{b}} = \frac{\partial E}{\partial \mathbf{z}}}$$

and, if needed, $\frac{\partial E}{\partial \mathbf{x}} = W^\top (\partial E / \partial \mathbf{z})$. For common activations: ReLU: $\sigma'(z) = \mathbf{1}_{z>0}$, tanh: $1 - \tanh^2(z)$, sigmoid: $\sigma(z)(1 - \sigma(z))$. Softmax with cross-entropy simplifies to $\partial E / \partial \text{logit} = \hat{\mathbf{p}} - \mathbf{y}$.



Why backprop is efficient. A naive "search all directions" (finite differences) needs one extra loss evaluation per parameter. Reverse-mode autodiff (backprop) uses the computation graph and the chain rule to obtain all partials in time comparable to one or a few forward passes.

4. Practical Notes

- **Learning rate η:** too large $\Rightarrow$ divergence; too small $\Rightarrow$ slow. Use schedulers or adaptive methods (Adam, etc.).

- **Mini-batching:** stochastic noise often helps generalization; full-batch is stable but slow.
- **Local minima/flat regions:** acceptable in practice; many modern nets find flat minima that generalize well.
- **Gradient check:** compare analytic grad vs. finite differences on small problems to verify correctness.

Key Insight. *Machine learning does not know the full function.* It discovers the loss surface by observing how errors change as parameters move, using gradients computed via backpropagation.

This handout is intentionally concise and self-contained for a 1–3 page mini-lecture.